BODA VMS AI 학습 도구 매뉴얼 (별책)

VMS.DeepLearning — 데이터셋 · 라벨링 · 딥러닝 모델 학습 · 모델 파일(ONNX) 내보내기

문서 버전 v1.3 (2026-09-18) · 대상 SW v1.40.1 이상 · 본편「BODA VMS 사용자 매뉴얼 v3.0」10장에 대응하는 별책

> **이 문서의 위치.** AI 학습 도구(VMS.DeepLearning)는 설치 파일의 "설치 구성 선택" 화면에서 **AI 학습 도구 포함** 을 체크한 PC 에만 설치되는 **선택 구성 요소** 입니다(기본 해제 — 라벨링·학습은 MLOps 플랫폼이 기본이고, MLOps 서버 없이 직접 학습하는 PC 만 체크합니다. 업그레이드 시 이전 선택 유지). 파이썬과 NVIDIA GPU 가 있는 학습용 PC 에서 사용하며, 검사만 수행하는 PC 에는 필요하지 않습니다. 만들어진 모델 파일을 검사에 쓰는 방법(비전 설정의 Detection · Classify · Anomaly · YOLOv8-seg · RF-DETR-seg 도구)은 본편 매뉴얼 §6.11 을, OCR 은 §6.7 을 참고하세요.

1. 딥러닝 모델 학습 (VMS.DeepLearning)

> ⚙ **이 장은 검사 모델을 만드는 엔지니어용입니다.** 현장에서 검사만 수행하는 작업자는 읽지 않아도 됩니다.

딥러닝 검사 도구(비전 설정의 Deep Learning 카테고리 — 본편 매뉴얼 §6.11)를 쓰려면 먼저 우리 제품의 사진으로 학습한 **모델 파일**이 있어야 합니다. VMS.DeepLearning 은 그 모델을 만드는 프로그램입니다 — 제품 사진을 모아 정답을 표시(라벨링)하고, 학습을 실행해서, 검사에 쓸 모델 파일(ONNX)을 만드는 과정을 한 화면에서 처리합니다.

비전 설정(VMS.VisionSetup) 화면 상단의 **[Deep Learning]** 버튼으로 실행합니다.

화면 구성

화면은 세 부분으로 나뉩니다 — 왼쪽은 데이터셋과 이미지 목록, 가운데는 사진 위에 정답을 그리는 캔버스, 오른쪽은 클래스 · 라벨 · 내보내기 · 학습 패널입니다. 각 섹션 제목과 입력란 라벨 옆의 파란 물음표(?) 아이콘에 마우스를 올리면 쉬운 설명 창이 열립니다.

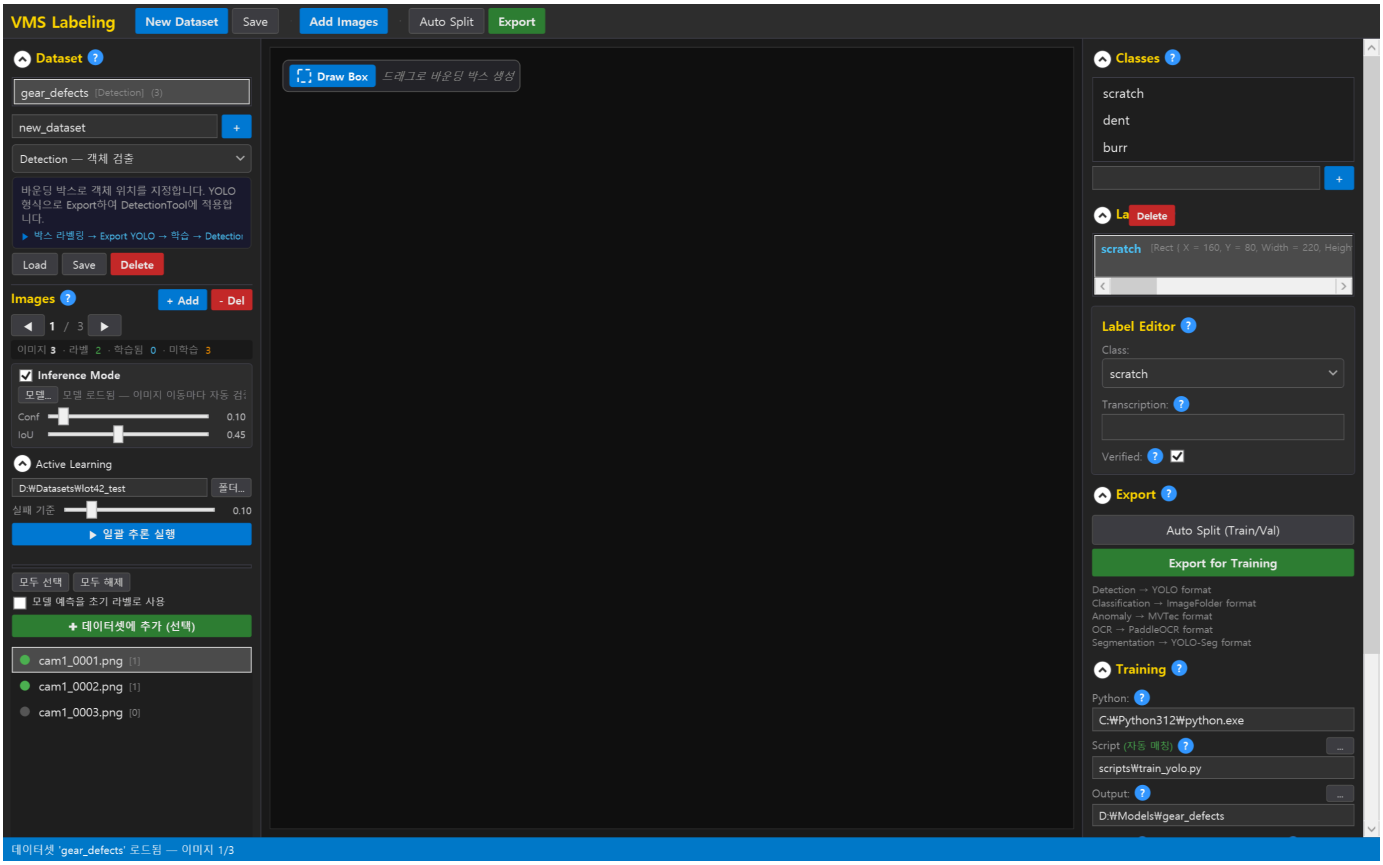


그림. VMS.DeepLearning — 데이터셋·이미지 목록(좌) / 라벨링 캔버스(중) / 클래스·라벨·학습(우) [Detection 데이터셋 예시]



그림. 상단 툴바 — [New Dataset] / [Save] / [Add Images] / [Auto Split] / [Export]

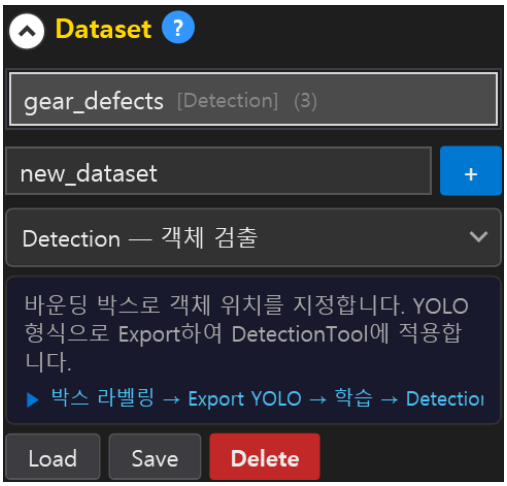


그림. Dataset 섹션 — 데이터셋 목록, 새 이름 + 작업 유형 선택, [Load]·[Save]·[Delete]

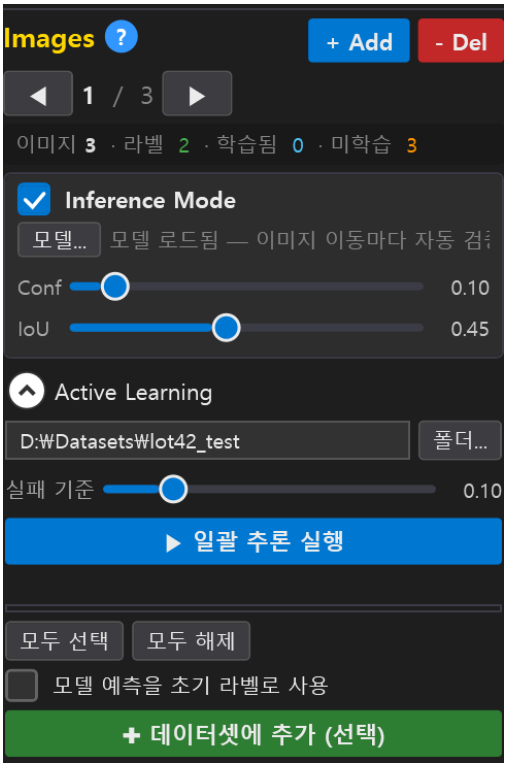


그림. Images 섹션 — [+ Add]·[- Del], 이미지 이동(◀/▶), 라벨·학습 현황 요약

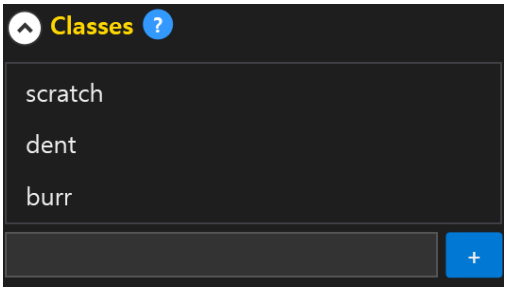


그림. Classes 섹션 — 클래스 목록과 추가([+])

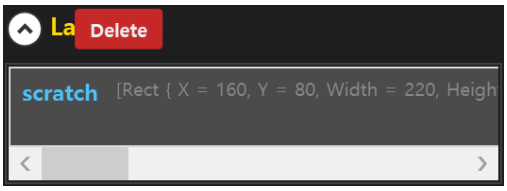


그림. Labels 섹션 — 현재 이미지의 라벨 목록, [Delete]

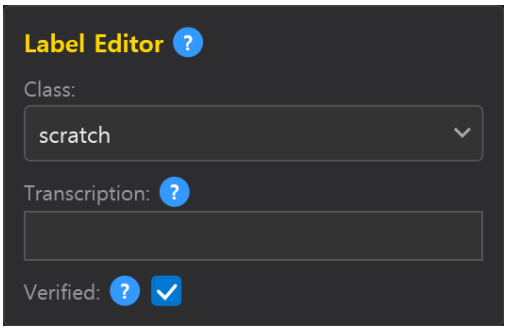


그림. Label Editor — 선택한 라벨의 클래스 / 텍스트(Transcription) / 검증 여부 편집

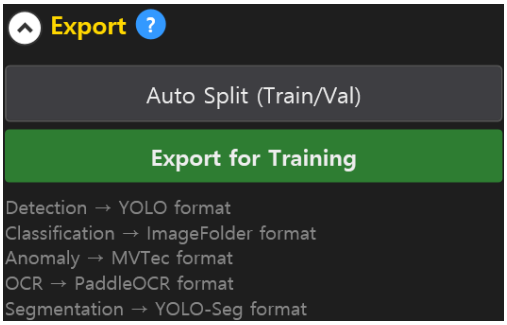


그림. Export 섹션 — [Auto Split (Train/Val)] / [Export for Training] + 작업 유형별 내보내기 형식 안내

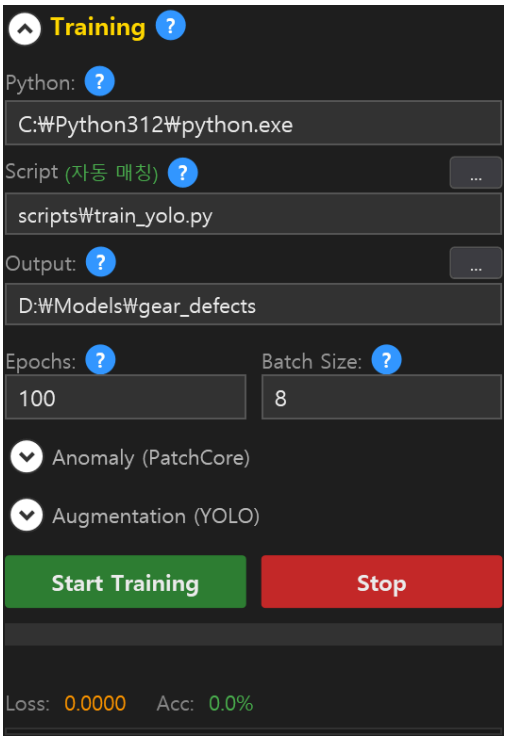


그림. Training 섹션 — Python·스크립트·출력 경로, Epochs/Batch Size, [Start Training]·[Stop], 진행률·로그

작업 유형 — 데이터셋을 만들 때 선택

무엇을 검사할지에 따라 다섯 가지 작업 유형 중 하나를 고릅니다. 유형마다 정답을 표시하는 방법이 다릅니다:

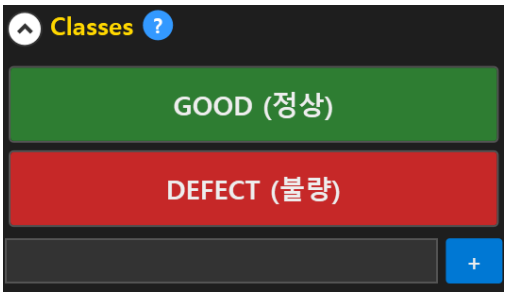


그림. Anomaly(이상 탐지) 유형의 분류 버튼 — [GOOD (정상)] / [DEFECT (불량)]

작업 유형	용도	정답 표시 방법
Detection (객체 검출)	결함이 어디 있는지 찾기	마우스 드래그로 결함 둘레에 상자를 그립니다
Classification (분류)	이미지 전체를 종류별로 구분	이미지마다 해당하는 클래스 버튼을 누릅니다
Anomaly (이상 탐지)	정상/불량 구분 — 불량 사진이 적을 때 유리	[GOOD (정상)] / [DEFECT (불량)] 버튼으로 분류합니다
OCR (문자 인식)	각인 · 라벨의 문자 읽기	문자 영역에 상자를 그리고 정답 텍스트를 입력합니다
Segmentation (분할)	객체의 정확한 윤곽까지 필요할 때	SAM 모델을 불러온 뒤 객체를 클릭하면 윤곽이 자동으로 생깁니다

라벨링에서 학습까지 — 기본 절차

1. 왼쪽 Dataset 섹션에서 이름을 입력하고 작업 유형을 고른 뒤 [+] 버튼으로 데이터셋을 만듭니다.
2. 위쪽 **[Add Images]** 버튼으로 검사 대상 사진들을 불러옵니다.
3. 사진마다 정답을 표시합니다 (방법은 위 표 참고). 키보드 ←/→ 로 이미지를 이동하며, 라벨이 완료된 이미지는 목록에 초록 점으로 표시됩니다.
4. **[Auto Split]** — 학습용/검증용 이미지를 자동으로 나눕니다.
5. **[Export]** — 학습에 필요한 형식으로 내보냅니다. 작업 유형에 맞는 형식이 자동으로 선택됩니다.
6. 오른쪽 Training 섹션에서 **[Start Training]** 을 누르면 학습이 시작됩니다. 진행률과 Loss(오차) · 정확도가 실시간으로 표시되고, [Stop] 으로 중단할 수 있습니다.
7. 학습이 끝나면 모델 파일(ONNX)이 만들어집니다. 이 파일을 비전 설정의 딥러닝 도구(Detection / Classify / Anomaly 등)에 지정하면 검사에 사용됩니다.

> **학습 환경:** 학습 기능은 Python 이 설치된 PC 에서 동작하며, NVIDIA GPU 가 있으면 훨씬 빠릅니다 (라벨링과 추론 확인은 GPU 없이도 가능). Python 경로는 Training 섹션에 표시되며, 학습 스크립트는 작업 유형에 맞게 자동으로 선택됩니다. 환경 준비는 설치 담당자에게 문의하세요.

학습한 모델을 라인에 내보내기 — [레지스트리에 등록]

MLOps 모델 레지스트리를 운영하는 현장에서는 학습이 끝난 모델 파일을 PC 마다 복사하지 않고 레지스트리에 올립니다. Training 섹션의 **[레지스트리에 등록]** 버튼을 누르면 창이 열립니다:

1. 웹 계정으로 로그인합니다 (생산 관리 웹의 계정 — 현재는 **관리자(Admin) 계정**만 등록할 수 있습니다).

2. 새 모델 계열을 만들거나 기존 계열을 고릅니다. 같은 제품 · 같은 검사의 모델은 한 계열에 버전으로 쌓입니다.

3. 라이선스와 메모를 적고 **[올리기]** 를 누릅니다. YOLO 계열 모델은 라이선스(AGPL-3.0) 표기가 필수입니다.

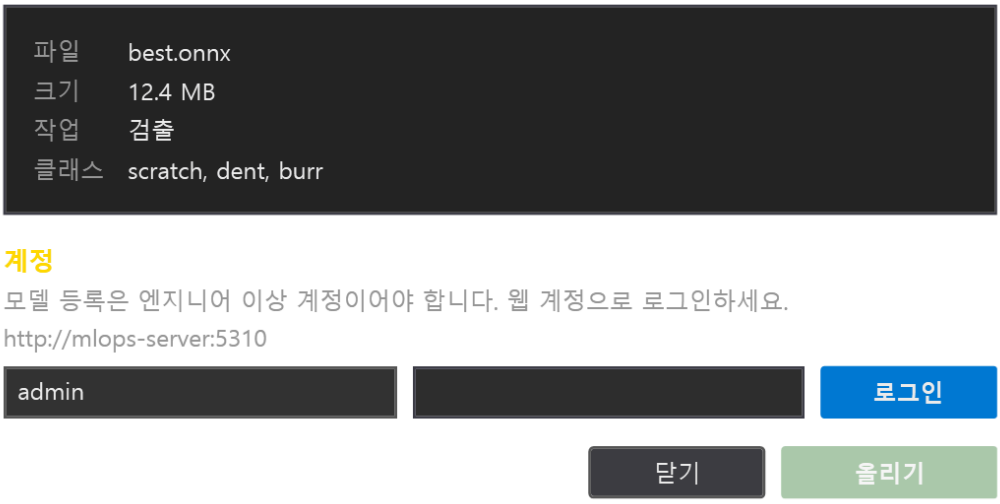


그림. [레지스트리에 등록] 창 — 로그인 전: 올릴 모델 파일 정보와 웹 계정 로그인

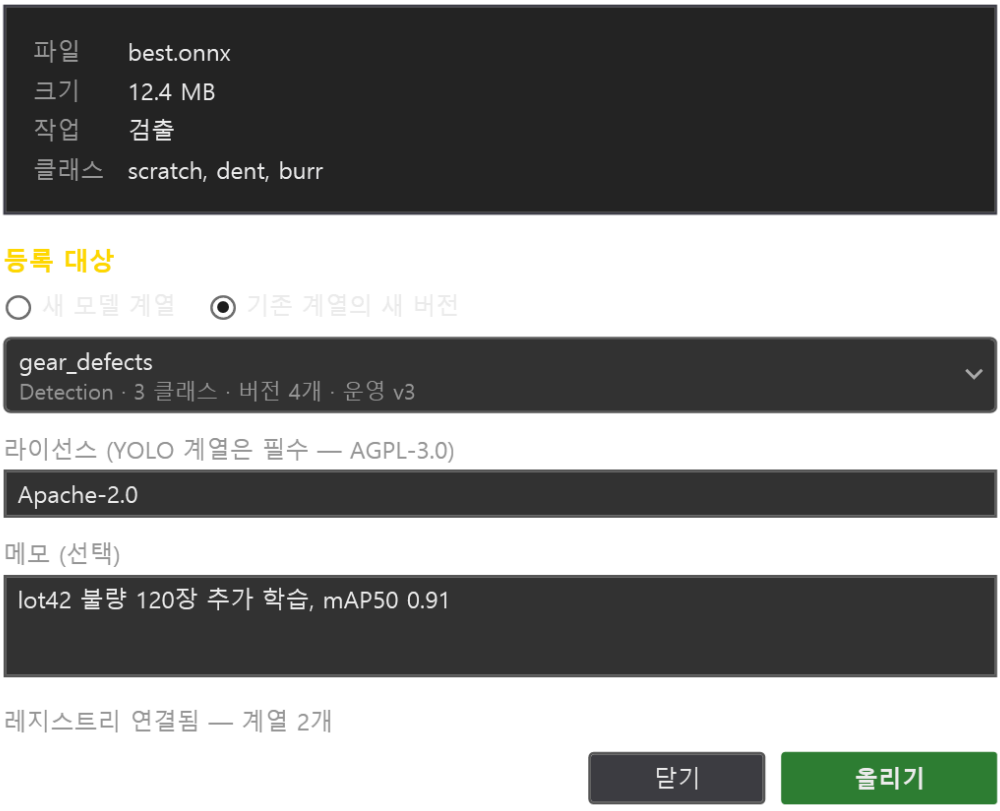


그림. [레지스트리에 등록] 창 — 로그인 후: 기존 계열을 골라 새 버전으로 올리기 직전

올라간 버전은 **후보(Candidate)** 상태라 라인에 바로 나가지 않습니다. MLOps 관리 화면에서 스테이징 → 운영으로 승격해야 라인 PC가 `model://` 참조로 받아 갑니다(본편 §9.10 레시피 바인딩). 이 창은 설정 마법사 2단계 고급 설정의 **MLOps 서버 주소** 와 **Web 서버 주소** 가 있어야 열리며, 없으면 무엇이 빠졌는지 알려 줍니다.

웹에서 라벨링한 데이터셋 받기 — [웹 데이터셋 내려받기]

라벨링을 웹(MLOps 데이터 관리 화면)에서 했다면 그 결과를 이 도구로 받아 학습할 수 있습니다. Export 섹션의 **[웹 데이터셋 내려받기]** 를 누르면 창이 열립니다:

1. 데이터셋을 고르고, 이미 떠 둔 **판(스냅샷)** 을 고르거나 **[새 판 뜨기]** 로 지금 상태를 굳힙니다. 학습은 항상 판 단위로 하므로 나중에 같은 판으로 재현할 수 있습니다.
2. 옵션 — **미라벨 포함**(검출 학습의 배경 샘플용) · **검토 완료만**(사람이 검토한 라벨만).
3. 받기가 끝나면 내보내기 폴더(`{판 이름}-{해시 8자리}`)가 만들어지고 학습 데이터셋 경로가 그 폴더로 자동 설정됩니다. 바로 [Start Training] 을 누르면 됩니다. 같은 판을 다시 받으면 폴더를 비우고 새로 풉니다.



그림. [웹 데이터셋 내려받기] 창 — 데이터셋과 판을 고르고 받기 직전

> **라인에서 올라온 불량 사진:** 웹의 데이터 관리 화면에 출처 "라인 NG" 로 쌓이는 사진은 검사 라인의 VMS 메인 → Image Save Settings 창의 "MLOps 학습 데이터 수집" 카드에서 켭니다(본편 §4.6). 라인 사진 → 웹 라벨링 → 이 도구로 받기 → 학습 → [레지스트리에 등록] → 승격 → 라인 적용의 한 바퀴가 이렇게 이어집니다.

학습 결과 바로 확인 (Inference Mode)

왼쪽의 **Inference Mode** 를 켜면 학습된 모델이 이미지를 넘길 때마다 자동으로 검사해서 결과 상자를 화면에 겹쳐 보여줍니다 — 학습이 잘 됐는지 그 자리에서 확인할 수 있습니다. 두 개의 슬라이더로 민감도를 조절합니다:

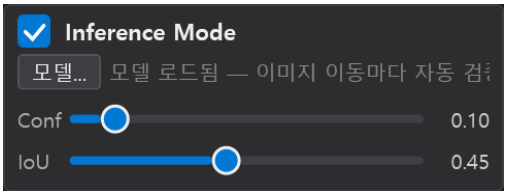


그림. Inference Mode — 모델 선택, Conf(신뢰도)·IoU(중복 제거) 슬라이더

- **Conf** (신뢰도 기준) — 이 값보다 확신이 낮은 검출은 표시하지 않습니다. 갓 학습한 모델은 낮게(0.05~0.15) 시작하는 것을 권장합니다.
- **IoU** (중복 제거 기준) — 같은 결함에 상자가 여러 개 겹칠 때 정리하는 강도입니다.

부족한 데이터 보강 (Active Learning)

모델을 효율적으로 키우는 기능입니다. 학습에 쓰지 않은 사진 폴더를 지정하고 **[일괄 추론 실행]** 을 누르면, 모델이 잘 못 맞히는 사진일수록 위에 오도록 골라 줍니다. 고른 사진을 **[데이터셋에 추가]** 로 가져와 라벨링하고 다시 학습하면, 모델이 약한 부분부터 보강됩니다. "모델 예측을 초기 라벨로 사용" 을 켜면 모델의 검출 결과가 라벨로 미리 채워져 검토만 하면 됩니다.

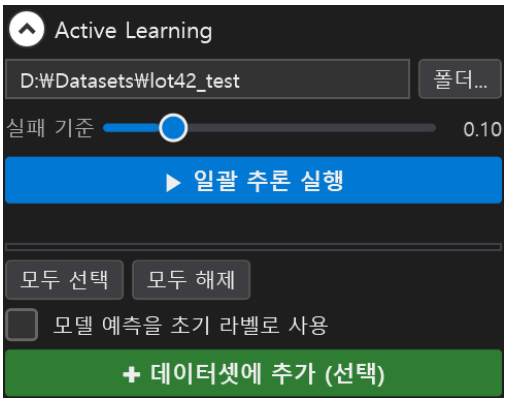


그림. Active Learning — 테스트 폴더 / 실패 기준 / [▶ 일괄 추론 실행] / [✚ 데이터셋에 추가]

클릭 한 번으로 윤곽 라벨링 (SAM)

Segmentation 데이터셋에서는 오른쪽 SAM Model 섹션에서 모델 파일 두 개 (Encoder/Decoder)를 지정하고 **[Load SAM Model]** 을 누릅니다. 그 다음 이미지에서 객체를 클릭만 하면 윤곽 마스크가 자동으로 만들어집니다:

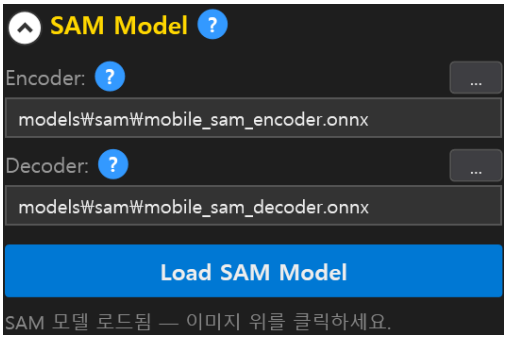


그림. SAM Model 섹션 — Encoder/Decoder 파일 지정, [Load SAM Model]

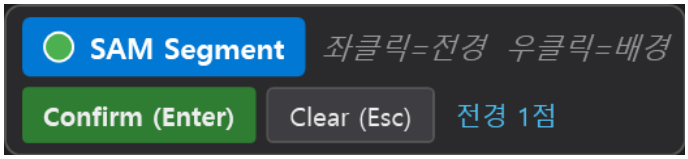


그림. SAM 라벨링 도구모음 — 좌클릭 전경 / 우클릭 배경, [Confirm (Enter)]·[Clear (Esc)]

- 왼쪽 클릭 — 포함할 부분 지정 / 오른쪽 클릭 — 제외할 부분 지정
- **Enter** — 미리보기 마스크를 라벨로 확정 / **Esc** — 취소

> **3D 검사에도 이 모델을 씁니다:** Segmentation 으로 학습한 모델은 3D 객체 검출 (본편 §7.9)의 첫 단계에서 그대로 사용됩니다. 3D 카메라를 쓰는 경우에도 학습은 언제나 **2D 사진** 으로 합니다 — 그 3D 카메라가 함께 찍는 2D 이미지를 모아 라벨링하세요.

2. 학습 환경 준비

> **MLOps 서버를 쓰는 현장은 이 장을 건너뛰어도 됩니다.** 학습을 웹에서 맡기는 구성이면 파이썬 준비가 필요 없습니다 — 아래 "웹에서 학습하기" 를 보세요. 이 장(§2.1~)은 *서버 없이 이 PC 에서 직접 학습*하는 경우의 준비입니다.

웹에서 학습하기 — 학습 PC 한 대로 모으기

라벨링은 웹에서 하고, 학습은 공장에 한 대 둔 **학습 PC(GPU)** 가 맡는 구성입니다. 라인 PC 는 종전처럼 검사(추론)만 합니다.

1. **학습 워커 설치** — 학습 PC 에서 `BODA-VMS-TrainWorker-<버전>.msi` 를 실행합니다. 설치 마법사에서 **관리 서버 주소**와 **워커 토큰**을 입력하면 서비스가 바로 시작되고, 파이썬 환경을 스스로 준비합니다(온라인 기준 10~40분, 약 6GB). 폐쇄망은 `-offline.msi` 를 씁니다. 워커 이름은 이 PC 이름이 기본값이고, GPU 가 없는 PC 에서 시험만 해 볼 때는 [GPU 없이 CPU 로만 동작 허용]을 체크합니다. 같은 화면에서 Python 3.12 설치 여부도 함께 알려 줍니다.

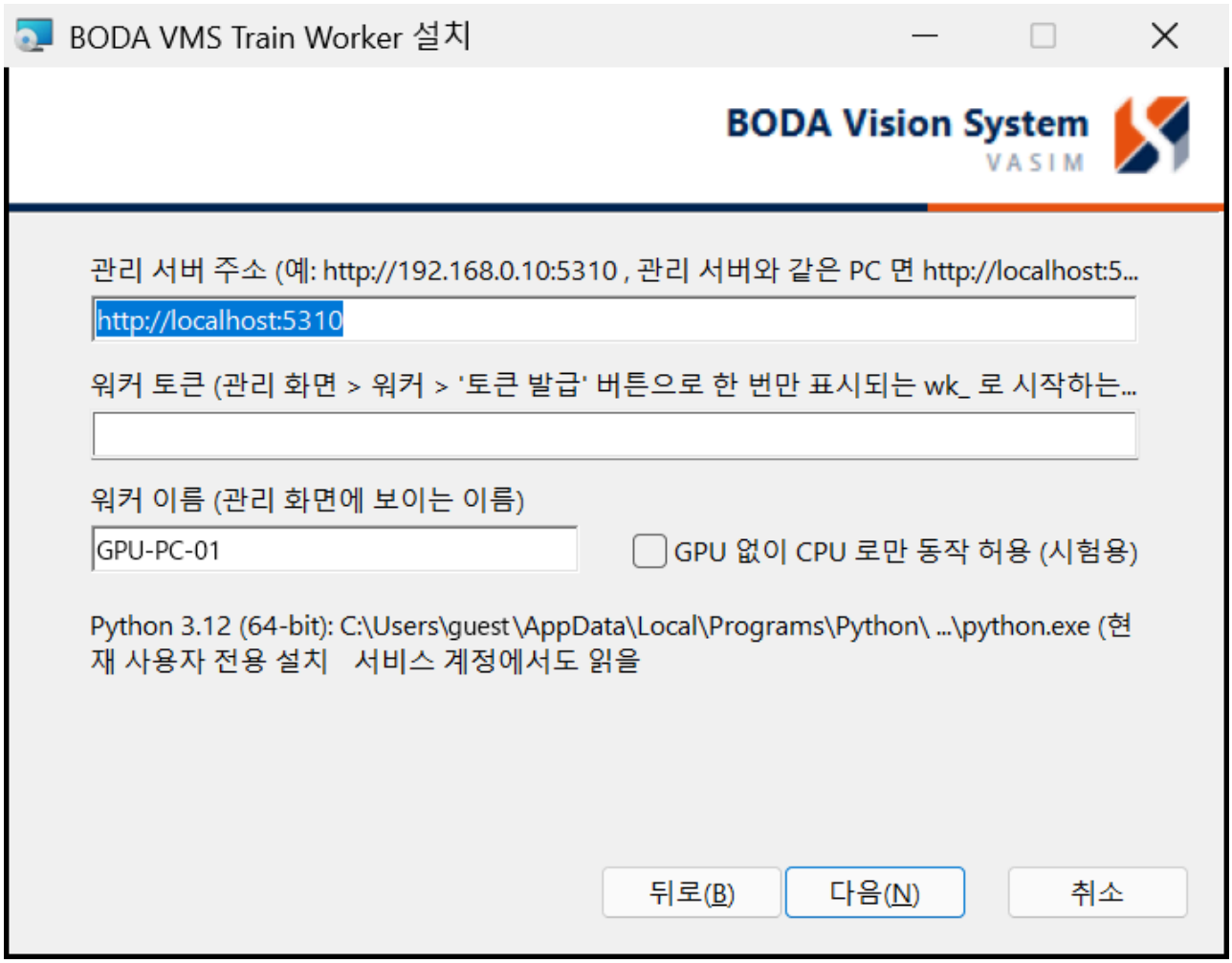


그림. 학습 워커 설치 마법사 — 관리 서버 주소 · 워커 토큰 · 워커 이름 · CPU 전용 옵션(Python 3.12 탐지 결과 표시)

1. **워커 토큰 발급** — 관리 화면 ▸ 워커 ▸ [+ 워커 등록] 에서 학습 PC 마다 하나씩 발급해 위 마법사에 붙여 넣습니다. 발급된 토큰은 그 자리에서 한 번만 보여 주므로 바로 복사해 둡니다.

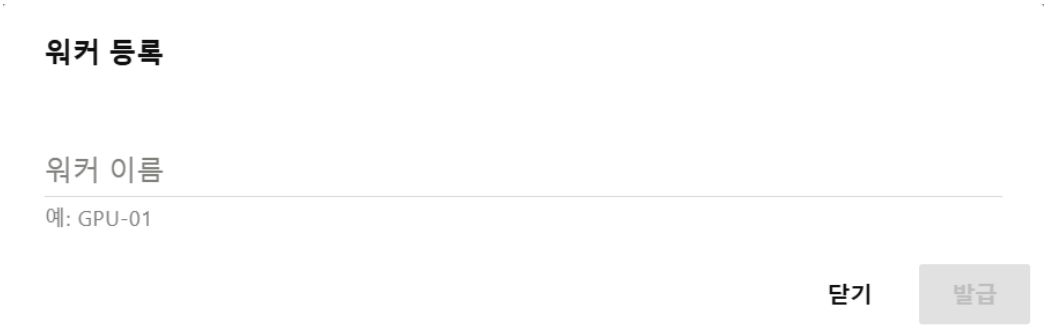


그림. 워커 등록 — 학습 PC 이름을 넣고 [발급] 하면 워커 토큰이 나온다

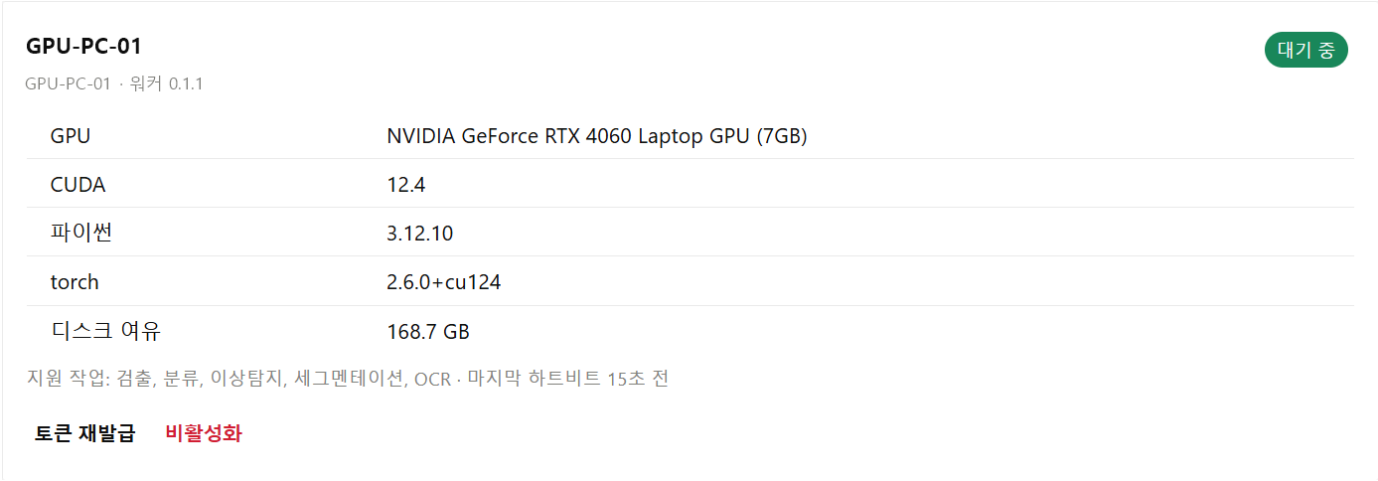


그림. 관리 화면 ▸ 워커 — 등록된 학습 PC 카드(GPU·CUDA·파이썬·torch·디스크 여유, 마지막 하트비트, 상태)

1. **학습 작업 제출** — 웹에서 데이터셋과 **사전학습 자산**을 고르고 작업을 제출하면 워커가 받아 학습합니다. 사전학습 자산을 고르지 않으면 워커가 학습을 시작하지 못합니다.

2. **결과 확인·승격** — 작업 상세에서 학습 곡선과 지표를 보고, 쓸 만하면 모델 버전을 승격합니다. 라인은 레시피의 모델 경로를 `model://` 참조로 두면 승격된 버전을 자동으로 받습니다(본편 §9.10).

학습 PC 와 라인 PC 는 **같은 MLOps 서버 주소**를 씁니다(설정 마법사 2단계 고급 설정, 본편 §3.3). 설치 구성 선택의 "AI 학습 도구 포함" 체크는 *MLOps 서버가 없는 현장*에서 이 PC 로 직접 학습할 때만 씁니다.

학습 기능(직접 학습)은 별도로 설치한 **Python 3.12** 와 파이썬 패키지로 동작합니다. 라벨링과 Inference Mode(학습 결과 확인)는 파이썬 없이도 됩니다. 아래 준비는 설치 담당자가 학습용 PC 에서 한 번만 합니다.

2.1 파이썬 패키지

작업 유형	설치 명령 (관리자 PowerShell)
Detection (객체 검출) — 기본 백본 D-FINE	`py -3.12 -m pip install torch torchvision "transformers>=4.52" onnx onnxruntime pyyaml` (선택) `py -3.12 -m pip install torchmetrics pycocotools` — 학습 패널에 검증 mAP50 표시
Segmentation (인스턴스 분할) — RF-DETR-seg	`py -3.12 -m pip install "rfdetr[train]>=1.10,<2" onnx onnxruntime` 세그멘테이션 데이터셋을 열면 학습 스크립트가 `train_rfdetr_seg.py` 로 자동 선택됩니다(COCO 형식 내보내기). 결과 모델은 비전 설정의 RF-DETR-seg 도구(본편 §6.11)에 지정합니다.
Classification (분류)	`py -3.12 -m pip install torch torchvision onnx`
Anomaly (이상 탐지)	`py -3.12 -m pip install "anomalib[core]>=1.1,<2" "pandas<3" "matplotlib<3.10" python-dotenv scikit-learn torch torchvision onnx` (anomalib 이 없으면 간이 PatchCore 로 자동 전환 — 두 방식 모두 VisionSetup 이상 탐지 도구가 그대로 읽는 ONNX 를 만듭니다)
OCR (문자 인식)	`py -3.12 -m pip install paddlepaddle paddleocr paddle2onnx onnx`
Segmentation 라벨링 보조 (SAM)	모델 파일 2개(Encoder/Decoder ONNX)만 있으면 됨 — 설치 담당자에게 요청

NVIDIA GPU 를 쓰려면 torch 는 CUDA 빌드(예: `--index-url https://download.pytorch.org/whl/cu124`)로 설치합니다. CPU 만으로도 학습은 되지만 매우 느립니다.

학습이 끝나면 생기는 파일

학습이 끝나면 출력 폴더에 모델 파일(`.onnx`) 말고도 두 가지가 함께 생깁니다 — 웹에서 학습하든 이 PC 에서 학습하든 같습니다.

- `metrics.json` — 에폭 수, 가장 좋았던 에폭, 학습·검증 손실, mAP50 같은 요약 숫자와 에폭별 기록입니다. 웹에 올리면 모델 버전 지표로 함께 보입니다.
- `curves.png` — 손실과 mAP50 곡선 그림입니다. 가장 좋았던 에폭이 점선으로 표시돼, 학습이 더 필요한지(곡선이 아직 내려가는 중인지) 한눈에 볼 수 있습니다.

2.2 Detection(D-FINE) 학습 시 알아둘 점

- **사전학습 모델** — Training 섹션의 사전학습 모델 입력란을 비워 두면 인터넷에서 자동으로 내려받습니다(`ustc-community/dfine-small-obj2coco`). 인터넷이 없는 PC 는 세 파일(`config.json`, `preprocessor_config.json`, `model.safetensors`)을 받아 둔 폴더 경로를 입력란에 넣습니다.
- **다운로드가 인증서 오류로 실패할 때** — 백신의 HTTPS 검사(예: Avast Web Shield)가 원인인 경우가 많습니다. `py -3.12 -m pip install truststore` 를 설치하면 Windows 인증서 저장소를 사용해 해결됩니다.

- **학습률** — D-FINE 권장값은 **0.00025** 입니다. 더 크면 학습 로그에 경고가 뜨고 발산할 수 있습니다.
- **증강** — hsv_h / hsv_s / hsv_v 는 적용되고, mosaic / mixup 은 YOLO 학습에만 적용됩니다(D-FINE 에서는 무시).
- **결과 파일** — 출력 폴더의 `best.onnx`. 본편 §6.11 의 Detection 도구에 지정하면 모델 종류(D-FINE / YOLO)를 자동으로 판별합니다. Input Size 는 학습 imgsz(기본 640)와 같게 둡니다.

3. 모델·라이선스 안내

AI 학습 도구가 기본으로 쓰는 학습 스택은 상업적 사용에 제한이 없는 라이선스입니다 — D-FINE(Apache 2.0) · RF-DETR(Apache 2.0) · HuggingFace transformers(Apache 2.0) · torchvision(BSD) · anomalib(Apache 2.0) · PaddleOCR(Apache 2.0) · MobileSAM(Apache 2.0). 학습으로 만든 모델 파일은 귀사의 자산으로 고객사에 자유롭게 배포할 수 있습니다.

> **Ultralytics YOLO(train_yolo.py) 주의.** Ultralytics YOLO 는 AGPL-3.0 이며, Ultralytics 는 학습한 가중치와 그 ONNX 변환본까지 파생물로 봅니다. 학습 스크립트 경로를 직접 `train_yolo.py` 로 바꾸는 것은 Ultralytics Enterprise License 를 보유한 사이트에서만 하세요. 그 외에는 기본값(D-FINE)을 사용합니다.